Notas de Estudo — Projeto de Rede Social (Python + SQLite)

1. Arquitetura do projeto

O fluxo de chamadas segue uma separação em camadas:

- `main.py` recebe os comandos do usuário.
- `rede.py` repassa as chamadas para o banco.
- `database.py` executa as operações no SQLite.

Essa divisão evita que a lógica de negócio (regras da "rede social") fique misturada com o SQL puro, e evita que `main.py` precise saber como o banco funciona por dentro.

---

2. Dúvidas resolvidas sobre SQLite e cursores

`conn.cursor()` é um ponteiro?

Não exatamente, mas a intuição não está longe.

Um cursor é mais como um **"operador"** que você usa para executar comandos SQL e navegar pelos resultados. Pense nele como a pessoa que vai até a estante (o banco) buscar ou guardar algo, usando a conexão (`conn`) como a chave da porta.

Não é um ponteiro de memória como em C — é um objeto Python que sabe "conversar" com o SQLite.

`DELETE FROM sqlite_sequence`

Quando `popular_banco.py` faz:

cursor.execute("DELETE FROM sqlite_sequence WHERE name='usuarios'")

Ele está dizendo ao SQLite: *"esqueça que você já gerou IDs até 5, comece do 1 de novo na próxima inserção."*

`.commit()`

É quem confirma as alterações, tirando-as do estado de "pendente".

> Analogia: é como escrever um documento e não apertar **Ctrl+S**. Sem o `commit()`, as alterações ficam perdidas (não são persistidas no banco).

---

3. O que faz `if __name__ == "__main__": main()`

Cada arquivo Python tem uma variável invisível chamada `__name__`.

- Quando você roda o arquivo diretamente (`python init_db.py`), o Python define `__name__` como `"__main__"`.
- Quando o arquivo é **importado** por outro (como `from init_db import criar_banco`), `__name__` vira o nome do arquivo (`"init_db"`), não `"__main__"`.

Esse `if` funciona como um filtro: *"só execute essa função automaticamente se este arquivo foi o que rodou direto, não se foi só importado por outro arquivo."*

É por isso que `main.py` pode fazer `from init_db import criar_banco` sem que `init_db.py` tente rodar sua própria lógica de novo por acidente.

---

4. `not` e inversão de lógica nos testes

Comparando as duas formas

```
# Do jeito que está (mais claro):
resultado = rede.criar_amizade(usuario_teste, usuario_teste)
checar("Bloqueio de autoamizade", not resultado)

# Alternativa (mais confusa):
resultado = not rede.criar_amizade(usuario_teste, usuario_teste)
checar("Bloqueio de autoamizade", resultado)
```

A vantagem da primeira forma é **não alterar a informação original** — `resultado` continua representando exatamente o que `criar_amizade` retornou. A inversão acontece só na hora da verificação (`checar`), deixando claro que estamos invertendo a lógica ali, e não escondendo isso dentro da variável.

Por que a negação confirma o resultado certo?

O comportamento correto e desejado é que o sistema **recuse** criar amizade consigo mesmo. Ou seja:

- `criar_amizade` deve retornar `False` (a operação falhou, como esperado).

- `not False` = `True` → **o teste passa** (✓)

A ideia-chave: **"sucesso do teste" é o oposto de "sucesso da operação"**. Quando o objetivo do teste é confirmar que algo *deveria* falhar, o teste só passa se a operação de fato falhar.

---

5. Outros pontos rápidos

- **`len()`**: faz a contagem de itens de uma lista (retorna o tamanho).
- **Correção de exibição dos perfis**: havia um "21a" e um "|" faltando na formatação — checar a string de exibição do perfil.

Mensagem de erro de username duplicado:
print("Username já existe. Repita o processo, por favor.")

-

---

6. Pontos em aberto / próximos passos

- [ ] Avaliar uso de `par()` para resolver a questão da **bidirecionalidade** (ex.: amizade entre A e B precisa valer nos dois sentidos) — identificar dificuldades específicas antes de implementar.
- [ ] Revisar uso de `unique` (constraint no banco) onde for aplicável.
- [ ] Esclarecer dúvida sobre **arrows / ponteiros** em Python (o que de fato existe e o que é só analogia, como já visto com `cursor`).
- [ ] Adicionar testes com **instrução para falha** — comentar dentro de `criar_amizade` os casos esperados de erro (ex.: autoamizade, amizade duplicada, usuário inexistente).
- [ ] Corrigir exibição dos perfis (item "21a" e "|" faltando).